



Universidad del Valle

SIMULACIÓN Y COMPUTACIÓN NUMÉRICA

Grupo 6

Profesora: María Patricia Trujillo

Informe Final

Simulación de Flujo Laminar Incompresible con Obstáculos

Discretización de Navier–Stokes 2D, Mallas, Método de Newton–Raphson, solución iterativa de sistemas lineales y funciones de interpolación (Spline cúbico natural)

Presentado por:

Juan David Rincón - 2342032
Maria Juliana Saavedra - 2344035
Libardo Alejandro Quintero - 2342181

27 de noviembre de 2025

Índice

1 Geometría, Mallas y Condiciones de Frontera	4
1.1 Configuraciones de Malla inicial supuesta	4
1.1.1 Dominio y Obstáculos	4
1.2 Malla Reducida con condiciones de frontera modificadas para el desarrollo (Actual):	4
1.2.1 Ubicación de las Vigas en la Malla Reducida	5
2 Discretización por Diferencias Finitas (DF): Paso a Paso	6
2.1 Paso 1: Ecuación de Partida Simplificada	6
2.2 Paso 2: Aproximación de las Derivadas	6
2.3 Paso 3: Sustitución en la Ecuación Continua	6
2.4 Paso 4: Aplicación de Parámetros de la Malla	7
2.5 Paso 5: Despeje de la Incógnita $u_{i,j}$	7
3 Las 9 Ecuaciones Especiales: Centro, Lados y Esquinas	8
3.0.1 Caso 1: Nodo Central (Interior Puro)	8
3.0.2 Caso 2: Lado Izquierdo (Inlet F)	8
3.0.3 Caso 3: Lado Derecho (Outlet H)	8
3.0.4 Caso 4: Lado Superior (Pared G)	8
3.0.5 Caso 5: Lado Inferior (Pared EA)	8
3.0.6 Caso 6: Esquina Superior Izquierda ($G \cap F$)	8
3.0.7 Caso 7: Esquina Superior Derecha ($G \cap H$)	8
3.0.8 Caso 8: Esquina Inferior Derecha ($EA \cap H$)	9
3.0.9 Caso 9: Esquina Inferior Izquierda ($EA \cap F$)	9
4 Método de Solución: Newton–Raphson	9
4.1 Geometría y Discretización del Dominio	9
4.2 Tabla de Parámetros de Simulación	9
4.3 Conversión Matriz-Vector	10
4.4 Inicialización Informada de $\mathbf{U}^{(0)}$	10
4.4.1 Generación de Intervalos Decrecientes	11
4.4.2 Asignación de Valores Iniciales	12
4.5 Cálculo del Vector de Residuales $\mathbf{F}(\mathbf{U})$	13
4.5.1 Ecuación Discretizada Central	13
4.5.2 Condiciones de Frontera	13
4.6 Cálculo de la Matriz Jacobiana $J(\mathbf{U})$	14
4.6.1 Derivadas Parciales para Nodos Internos	14
4.6.2 Derivadas Parciales para Nodos de Frontera	15
4.6.3 Estructura de la Matriz Jacobiana	15
4.6.4 Mapa de calor del Jacobiano	15
4.7 Proceso Iterativo de Newton-Raphson	16
4.8 Criterios de Parada	16
5 Solución del Sistema Lineal: Elección del Método Iterativo	18
5.1 Marco Teórico: La Teoría de Separación (Splitting)	18
5.1.1 Teorema de Convergencia	18
5.2 Métodos Analizados y Resultados	18

5.2.1	Método de Richardson	18
5.2.2	Método de Jacobi	19
5.2.3	Método de Gauss-Seidel	19
5.2.4	Método de Gradiente Descendente	19
5.3	Resumen de Métodos Analizados	20
5.4	El Problema con la Matriz Jacobiana	20
5.5	La Solución: Transformación a Sistema Simétrico Definido Positivo	20
5.5.1	Propiedades de $A = J^T J$	21
5.5.2	Efecto en el Número de Condición	21
5.6	Método de Gradiente Conjugado	21
5.6.1	Definición de Direcciones Conjugadas	21
5.6.2	Algoritmo de Gradiente Conjugado	22
5.6.3	Ventajas del Gradiente Conjugado	22
5.7	Criterios de Parada del Gradiente Conjugado	22
5.8	Resultados de la Implementación	22
5.9	Implementación y Resultados	23
5.9.1	Mapa de calor con valores numéricos	23
5.9.2	Mapa de calor	24
5.10	Refinamiento Visual mediante Interpolación	24
5.10.1	Metodología de Interpolación	24
5.10.2	Visualización de la Función de Interpolación	24
5.10.3	Resultado Final Suavizado	26
5.11	Observaciones sobre la Convergencia	27
6	Conclusiones y Próximos Pasos	28
7	Repositorio del Proyecto	28
7.1	Contenido del Repositorio	29
7.2	Sobre el Modelo y los Datos Iniciales	29
7.3	Documentación en el README	29

Resumen

Se presenta la implementación completa de una simulación numérica de flujo laminar incompresible en 2D alrededor de dos vigas sumergidas, basada en el problema propuesto por Landau & Páez. Partiendo del sistema de Navier–Stokes estacionario simplificado ($\nu = 1$, $\nabla P = 0$), se desarrolla la discretización por diferencias finitas centradas en malla uniforme ($h = 8$), derivando las **nueve ecuaciones especiales** para el tratamiento de fronteras (cuatro lados, cuatro esquinas y condición de no deslizamiento en vigas).

El sistema no lineal resultante (364 ecuaciones) se resuelve mediante el **método de Newton–Raphson**, con una estrategia de inicialización informada que refleja la física del flujo (velocidades decrecientes de entrada a salida). Para resolver el sistema lineal en cada iteración, se analizaron cinco métodos iterativos: *Richardson*, *Jacobi*, *Gauss-Seidel*, *Gradiente Descendente* y *Gradiente Conjugado*. El análisis del radio espectral demostró que los primeros cuatro métodos no convergen para nuestra matriz Jacobiana (no simétrica, no diagonalmente dominante).

La solución adoptada consiste en transformar el sistema $J \cdot H = -F$ al sistema equivalente $J^T J \cdot H = -J^T F$, donde $J^T J$ es simétrica y definida positiva (SDP), permitiendo aplicar el **Gradiente Conjugado**. Aunque el número de condición aumenta ($\kappa(J) \approx 24 \rightarrow \kappa(J^T J) \approx 577$), el método converge en ~ 100 iteraciones por paso de Newton, alcanzando convergencia total en ~ 116 iteraciones de Newton–Raphson.

Finalmente, se implementó **interpolación con splines cúbicos bidimensionales** (RectBivariateSpline de SciPy) para generar visualizaciones suavizadas del campo de velocidades, incluyendo superficies 3D y perfiles de corte. El código Python resultante genera automáticamente valores iniciales y produce mapas de calor, visualizaciones del Jacobiano y campos de velocidades refinados.

1 Geometría, Mallas y Condiciones de Frontera

1.1. Configuraciones de Malla inicial supuesta

1.1.1. Dominio y Obstáculos

El dominio computacional es un canal rectangular Ω con dos vigas sólidas internas (A y B). Las condiciones de velocidad u en las fronteras del canal son:

- **Izquierda (Inlet F):** Perfil de entrada impuesto, $u = V_0$.
- **Superior (Pared G):** Condición de pared, $u = V_0$.
- **Derecha (Outlet H):** Perfil de salida, $u = 0$.
- **Inferior (Pared EA):** Condición de pared, $u = 0$.

Sobre las superficies de las vigas A y B se impone la condición de no deslizamiento, es decir, $u = 0$ y $v = 0$.

Se trabajan dos escalas de malla para facilitar el desarrollo:

1. **Malla Original (Referencia):** Dimensiones $N_x = 400$, $N_y = 40$ con paso $h = 1$. Las vigas se ubican en las coordenadas físicas:

$$\text{Viga A: } x \in [176, 232], \ y \in [1, 16], \quad \text{Viga B: } x \in [296, 400], \ y \in [32, 40].$$

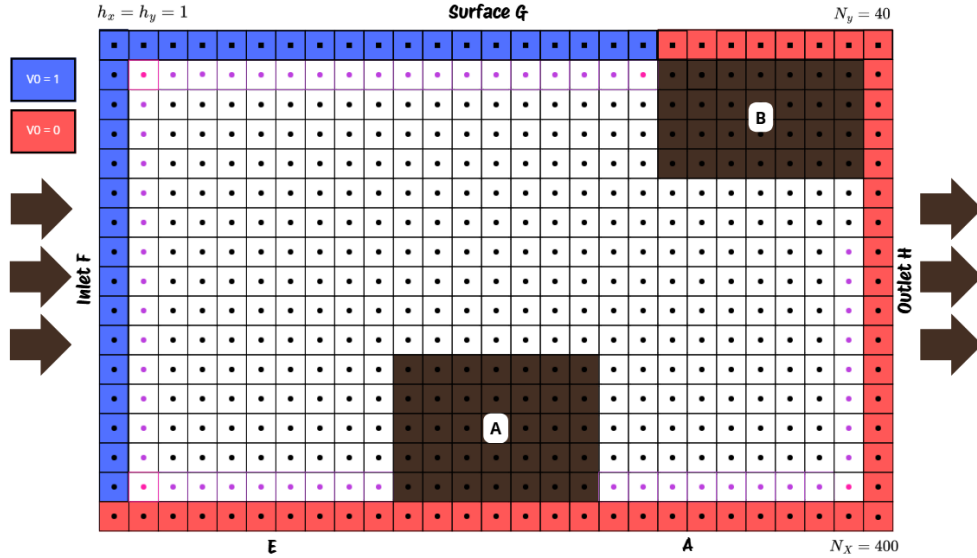


Figura 1: Esquema del dominio computacional inicial, mostrando la malla, la ubicación de las vigas A y B, y las condiciones de frontera.

1.2. Malla Reducida con condiciones de frontera modificadas para el desarrollo (Actual):

Para acelerar las pruebas, la malla se comprime por un factor $s = 8$, resultando en $N'_x = 50$ y $N'_y = 5$ puntos interiores. Esta versión a escala ($h = 8$) permite validar el código y los algoritmos de forma eficiente.

Modificación de fronteras: A diferencia de la malla original, en esta versión la frontera superior tiene $u = 0$ (en lugar de V_0), simplificando el problema para la fase de desarrollo. El resto de condiciones de frontera y la condición de no deslizamiento en las vigas se mantienen igual.

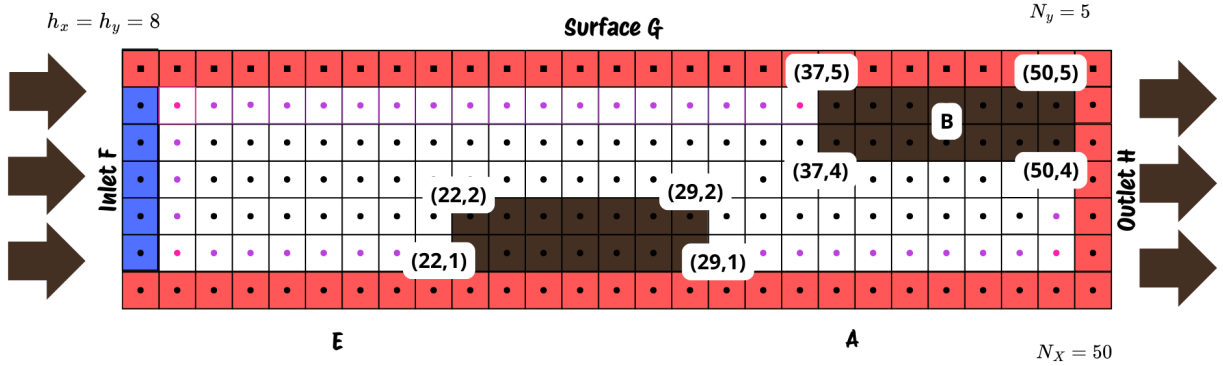
1.2.1. Ubicación de las Vigas en la Malla Reducida

Con el factor de escalamiento $s = 8$ y el espaciado de malla $h = 8$, las vigas se ubican en las siguientes coordenadas físicas en la malla reducida actual:

Viga A (Inferior): $x \in [176, 232]$, $y \in [8, 120]$, Viga B (Superior): $x \in [296, 400]$, $y \in [32, 40]$.

Estas coordenadas físicas corresponden a las siguientes posiciones en índices de malla (considerando que cada celda tiene tamaño $h = 8$):

- **Viga A (Inferior):** Columnas $j \in [22, 29]$, Filas $i \in [1, 2]$
- **Viga B (Superior):** Columnas $j \in [37, 50]$, Filas $i \in [4, 5]$



2 Discretización por Diferencias Finitas (DF): Paso a Paso

El núcleo del método numérico consiste en transformar la ecuación diferencial parcial continua en un sistema de ecuaciones algebraicas que pueden ser resueltas en una computadora. Este proceso se realiza mediante la discretización del dominio y de las derivadas. A continuación, se detalla el procedimiento paso a paso.

2.1. Paso 1: Ecuación de Partida Simplificada

Partimos de la ecuación de momento en la dirección x (NSx), que describe el balance de fuerzas en el fluido:

$$\nu (\partial_{xx}u + \partial_{yy}u) = u \partial_x u + v \partial_y u + \frac{1}{\rho} \partial_x P$$

Aplicando las simplificaciones establecidas para esta fase del proyecto (viscosidad cinemática $\nu = 1$ y gradiente de presión nulo $\partial_x P = 0$), la ecuación a discretizar se reduce a:

$$\partial_{xx}u + \partial_{yy}u = u \partial_x u + v \partial_y u \quad (1)$$

Esta ecuación balancea los términos de difusión (izquierda) con los términos convectivos (derecha).

2.2. Paso 2: Aproximación de las Derivadas

El siguiente paso es reemplazar cada derivada parcial por su aproximación en diferencias finitas centradas en un nodo genérico (i, j) de la malla. Estas aproximaciones tienen un error de segundo orden, $\mathcal{O}(h^2)$, lo que las hace adecuadas para obtener resultados precisos.

- **Términos de difusión (segundas derivadas):**

$$\begin{aligned} \partial_{xx}u|_{i,j} &\approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} \\ \partial_{yy}u|_{i,j} &\approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} \end{aligned}$$

- **Términos convectivos (primeras derivadas):**

$$\begin{aligned} \partial_x u|_{i,j} &\approx \frac{u_{i+1,j} - u_{i-1,j}}{2h_x} \\ \partial_y u|_{i,j} &\approx \frac{u_{i,j+1} - u_{i,j-1}}{2h_y} \end{aligned}$$

2.3. Paso 3: Sustitución en la Ecuación Continua

Ahora, sustituimos estas expresiones discretas en la ecuación continua simplificada (eq. (1)):

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} = u_{i,j} \left(\frac{u_{i+1,j} - u_{i-1,j}}{2h_x} \right) + v_{i,j} \left(\frac{u_{i,j+1} - u_{i,j-1}}{2h_y} \right)$$

Esta ecuación es la representación algebraica completamente discreta de nuestra ecuación diferencial.

2.4. Paso 4: Aplicación de Parámetros de la Malla

Para este proyecto, en la malla de desarrollo actual, se establece que el espaciado es uniforme con $h_x = h_y = h = 8$. Al multiplicar toda la ecuación por h^2 , el coeficiente de los términos convectivos se convierte en:

$$\text{Coeficiente} = \frac{h^2}{2h} = \frac{h}{2} = \frac{8}{2} = 4$$

La ecuación balanceada queda:

$$(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) + (u_{i,j+1} - 2u_{i,j} + u_{i,j-1}) = 4u_{i,j}(u_{i+1,j} - u_{i-1,j}) + 4v_{i,j}(u_{i,j+1} - u_{i,j-1}) \quad (2)$$

Agrupando los términos $u_{i,j}$ del lado izquierdo, obtenemos el factor $-4u_{i,j}$.

2.5. Paso 5: Despeje de la Incógnita $u_{i,j}$

El objetivo final es obtener una expresión explícita para $u_{i,j}$. Reorganizamos para despejar el término central $4u_{i,j}$, moviendo la suma de vecinos y restando los términos convectivos:

$$4u_{i,j} = (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) - 4u_{i,j}(u_{i+1,j} - u_{i-1,j}) - 4v_{i,j}(u_{i,j+1} - u_{i,j-1})$$

Finalmente, dividimos por 4 para obtener la fórmula de iteración.

$$u_{i,j} = \frac{1}{4} \left[(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) - 4u_{i,j}(u_{i+1,j} - u_{i-1,j}) - 4v_{i,j}(u_{i,j+1} - u_{i,j-1}) \right]. \quad (3)$$

Esta ecuación es la piedra angular del modelo numérico para la malla escalada.

3 Las 9 Ecuaciones Especiales: Centro, Lados y Esquinas

La fórmula eq. (3) se adapta para los bordes sustituyendo los vecinos "fantasma" por las condiciones de frontera conocidas (ver Figura de la malla en la sección anterior).

3.0.1. Caso 1: Nodo Central (Interior Puro)

Se utiliza la Ecuación 3:

$$u_{i,j} = \frac{1}{4} \left[(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) - 4u_{i,j}(u_{i+1,j} - u_{i-1,j}) - 4v_{i,j}(u_{i,j+1} - u_{i,j-1}) \right]$$

3.0.2. Caso 2: Lado Izquierdo (Inlet F)

Vecino izquierdo $u_{i-1,j} = V_0$.

$$u_{1,j} = \frac{1}{4} \left[(u_{2,j} + V_0 + u_{1,j+1} + u_{1,j-1}) - 4u_{1,j}(u_{2,j} - V_0) - 4v_{1,j}(u_{1,j+1} - u_{1,j-1}) \right]$$

3.0.3. Caso 3: Lado Derecho (Outlet H)

Vecino derecho $u_{i+1,j} = 0$.

$$u_{N_x,j} = \frac{1}{4} \left[(0 + u_{N_x-1,j} + u_{N_x,j+1} + u_{N_x,j-1}) - 4u_{N_x,j}(0 - u_{N_x-1,j}) - 4v_{N_x,j}(u_{N_x,j+1} - u_{N_x,j-1}) \right]$$

3.0.4. Caso 4: Lado Superior (Pared G)

Vecino superior $u_{i,j+1} = V_0$.

$$u_{i,N_y} = \frac{1}{4} \left[(u_{i+1,N_y} + u_{i-1,N_y} + V_0 + u_{i,N_y-1}) - 4u_{i,N_y}(u_{i+1,N_y} - u_{i-1,N_y}) - 4v_{i,N_y}(V_0 - u_{i,N_y-1}) \right]$$

3.0.5. Caso 5: Lado Inferior (Pared EA)

Vecino inferior $u_{i,j-1} = 0$.

$$u_{i,1} = \frac{1}{4} \left[(u_{i+1,1} + u_{i-1,1} + u_{i,2} + 0) - 4u_{i,1}(u_{i+1,1} - u_{i-1,1}) - 4v_{i,1}(u_{i,2} - 0) \right]$$

3.0.6. Caso 6: Esquina Superior Izquierda ($G \cap F$)

$U_L = V_0, U_T = V_0$.

$$u_{1,N_y} = \frac{1}{4} \left[(u_{2,N_y} + V_0 + V_0 + u_{1,N_y-1}) - 4u_{1,N_y}(u_{2,N_y} - V_0) - 4v_{1,N_y}(V_0 - u_{1,N_y-1}) \right]$$

3.0.7. Caso 7: Esquina Superior Derecha ($G \cap H$)

$U_R = 0, U_T = V_0$.

$$u_{N_x,N_y} = \frac{1}{4} \left[(0 + u_{N_x-1,N_y} + V_0 + u_{N_x,N_y-1}) - 4u_{N_x,N_y}(0 - u_{N_x-1,N_y}) - 4v_{N_x,N_y}(V_0 - u_{N_x,N_y-1}) \right]$$

3.0.8. Caso 8: Esquina Inferior Derecha (EA∩H)

$$U_R = 0, U_B = 0.$$

$$u_{N_x,1} = \frac{1}{4} \left[(0 + u_{N_x-1,1} + u_{N_x,2} + 0) - 4u_{N_x,1}(0 - u_{N_x-1,1}) - 4v_{N_x,1}(u_{N_x,2} - 0) \right]$$

3.0.9. Caso 9: Esquina Inferior Izquierda (EA∩F)

$$U_L = V_0, U_B = 0.$$

$$u_{1,1} = \frac{1}{4} \left[(u_{2,1} + V_0 + u_{1,2} + 0) - 4u_{1,1}(u_{2,1} - V_0) - 4v_{1,1}(u_{1,2} - 0) \right]$$

Nodos dentro de vigas (no-slip): Para cualquier nodo (i, j) que se encuentre dentro de las vigas A o B, se impone directamente $u_{i,j} = 0$ y $v_{i,j} = 0$.

4 Método de Solución: Newton–Raphson

El conjunto de ecuaciones discretas forma un sistema algebraico no lineal de gran tamaño. Para resolverlo, se utiliza el método de Newton–Raphson, que busca la raíz del vector de residuales $\mathbf{F}(\mathbf{U}) = 0$. Cada iteración consiste en:

$$J(\mathbf{U}^{(k)}) \Delta \mathbf{U}^{(k)} = -\mathbf{F}(\mathbf{U}^{(k)}), \quad \mathbf{U}^{(k+1)} = \mathbf{U}^{(k)} + \Delta \mathbf{U}^{(k)}.$$

Alternativamente, el método puede expresarse como:

$$\mathbf{U}^{(k+1)} = \mathbf{U}^{(k)} - J^{-1}(\mathbf{U}^{(k)}) \cdot \mathbf{F}(\mathbf{U}^{(k)})$$

4.1. Geometría y Discretización del Dominio

La geometría del problema (canal con dos bloques sólidos) se mostró en la sección de mallas. Para la simulación, se utiliza una malla con las siguientes características:

- Dimensiones del dominio: $L_x = 400$ unidades, $L_y = 40$ unidades
- Espaciamiento de malla: $h_x = h_y = h = 8$ unidades
- Número de nodos internos: $N_x = 50$ nodos en dirección x , $N_y = 5$ nodos en dirección y
- Total de nodos: $(N_x + 2) \times (N_y + 2) = 52 \times 7 = 364$ nodos

4.2. Tabla de Parámetros de Simulación

Para referencia rápida, la Tabla 1 resume todos los parámetros numéricos y físicos utilizados en la simulación actual.

Parámetro	Símbolo	Valor
Parámetros Geométricos		
Longitud del dominio	L_x	400
Altura del dominio	L_y	40
Espaciado de malla	h	8
Nodos en dirección x	N_x	50
Nodos en dirección y	N_y	5
Total de nodos	N_{total}	364
Parámetros Físicos		
Velocidad de entrada	V_0	1
Velocidad vertical	V_y	-0.000005
Viscosidad cinemática	ν	1
Densidad	ρ	constante
Parámetros Numéricos		
Tolerancia residual	ε_F	10^{-10}
Tolerancia cambio	δ_{step}	10^{-8}
Iteraciones máximas Newton	$k_{\text{máx}}$	200
Tolerancia gradiente conjugado	tol_{CG}	10^{-10}
Factor refinamiento visual	f	10

Cuadro 1: Parámetros numéricos y físicos utilizados en la simulación con malla reducida.

La malla se estructura como una matriz de velocidades:

$$\mathbf{U} = \begin{bmatrix} v_{0,0} & v_{0,1} & v_{0,2} & \cdots & v_{0,50} & v_{0,51} \\ v_{1,0} & v_{1,1} & v_{1,2} & \cdots & v_{1,50} & v_{1,51} \\ v_{2,0} & v_{2,1} & v_{2,2} & \cdots & v_{2,50} & v_{2,51} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ v_{6,0} & v_{6,1} & v_{6,2} & \cdots & v_{6,50} & v_{6,51} \end{bmatrix}$$

4.3. Conversión Matriz-Vector

Para aplicar el método de Newton-Raphson, la matriz de velocidades se convierte en un vector unidimensional. La transformación se realiza mediante:

$$\text{índice vectorial} = i \times N_{\text{cols}} + j$$

donde i es el índice de fila, j es el índice de columna, y $N_{\text{cols}} = 52$.

De forma inversa:

$$\text{fila} = \left\lfloor \frac{\text{índice}}{52} \right\rfloor, \quad \text{columna} = \text{índice} \bmod 52$$

El vector resultante tiene dimensión $N_{\text{total}} = 364$ ecuaciones.

4.4. Inicialización Informada de $\mathbf{U}^{(0)}$

Una buena estimación inicial es clave para la convergencia del método de Newton-Raphson. Se diseña una estrategia que refleja la física esperada del flujo (disminución de velocidad de izquierda a derecha y de arriba a abajo).

4.4.1. Generación de Intervalos Decrecientes

La malla se divide en secciones tanto horizontal como verticalmente. Para la malla reducida (50×5):

División Horizontal (Eje X): Se define $d_h = 10$ divisiones horizontales. El paso entre valores es:

$$\Delta_h = \frac{1}{2 \times d_h} = \frac{1}{20} = 0.05$$

Esto genera una lista de valores decrecientes:

$$L_h = [0.95, 0.90, 0.85, 0.80, 0.75, 0.70, 0.65, 0.60, 0.55, 0.50, 0.45, 0.40, 0.35, 0.30, 0.25, 0.20, 0.15, 0.10, 0.05, 0.00]$$

Esta lista contiene $2 \times d_h = 20$ valores, agrupados en 10 pares de intervalos:

Sección	Par de valores	Intervalo
1	0.95, 0.90	[0.90, 0.95]
2	0.85, 0.80	[0.80, 0.85]
3	0.75, 0.70	[0.70, 0.75]
4	0.65, 0.60	[0.60, 0.65]
5	0.55, 0.50	[0.50, 0.55]
6	0.45, 0.40	[0.40, 0.45]
7	0.35, 0.30	[0.30, 0.35]
8	0.25, 0.20	[0.20, 0.25]
9	0.15, 0.10	[0.10, 0.15]
10	0.05, 0.00	[0.00, 0.05]

Cuadro 2: Intervalos horizontales para valores iniciales.

Cada columna j de la malla se asigna a una sección horizontal. Como hay 50 columnas internas y 10 secciones:

$$\text{tamaño_sección_horizontal} = \frac{N_x}{d_h} = \frac{50}{10} = 5 \text{ columnas por sección}$$

División Vertical (Eje Y): Se define $d_v = 5$ divisiones verticales. El paso entre valores es:

$$\Delta_v = \frac{1}{2 \times d_v} = \frac{1}{10} = 0.1$$

Esto genera una lista de valores decrecientes:

$$L_v = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]$$

Esta lista contiene $2 \times d_v = 10$ valores, agrupados en 5 pares de intervalos:
Como hay 5 filas internas y 5 secciones:

$$\text{tamaño_sección_vertical} = \frac{N_y}{d_v} = \frac{5}{5} = 1 \text{ fila por sección}$$

Fila	Par de valores	Intervalo
1	0.9, 0.8	[0.8, 0.9]
2	0.7, 0.6	[0.6, 0.7]
3	0.5, 0.4	[0.4, 0.5]
4	0.3, 0.2	[0.2, 0.3]
5	0.1, 0.0	[0.0, 0.1]

Cuadro 3: Intervalos verticales para valores iniciales.

4.4.2. Asignación de Valores Iniciales

Para cada nodo interno (i, j) :

1. **Aporte Horizontal:** Se selecciona aleatoriamente uno de los dos valores del par correspondiente a la sección horizontal del nodo:

$$v_h = \text{random.choice}(L_h[m-1], L_h[m])$$

donde m es el índice del par en la lista horizontal.

2. **Aporte Vertical:** Se selecciona aleatoriamente uno de los dos valores del par correspondiente a la sección vertical del nodo:

$$v_v = \text{random.choice}(L_v[k-1], L_v[k])$$

donde k es el índice del par en la lista vertical.

3. **Valor Inicial Combinado:** El valor inicial en cada nodo es el **promedio** de su aporte horizontal y vertical:

$$v_{i,j}^{(0)} = \frac{v_h + v_v}{2}$$

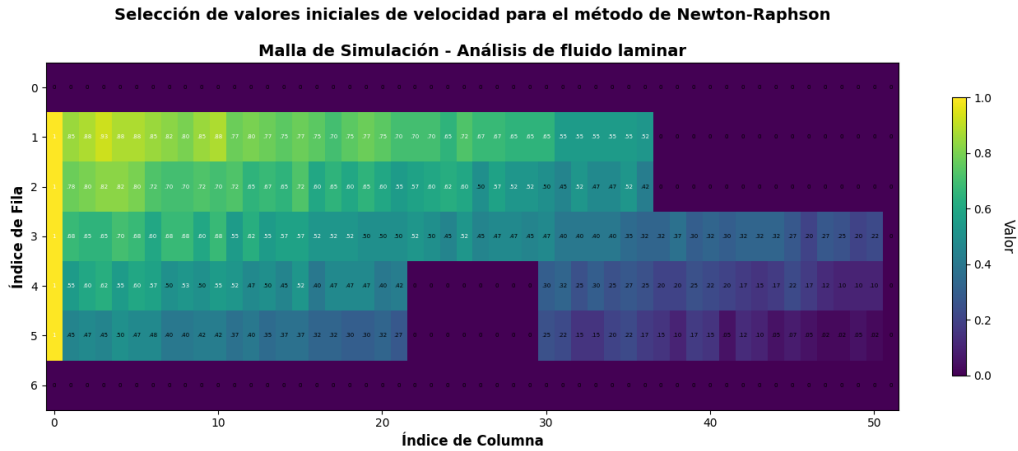


Figura 2: Malla de valores iniciales generada con el algoritmo de selección informada. Los colores representan la magnitud de la velocidad.

Esta estrategia garantiza que:

- Los valores disminuyen gradualmente de izquierda a derecha (simulando la pérdida de velocidad por fricción).
- Los valores disminuyen de arriba hacia abajo (reflejando el gradiente de velocidad típico).
- Hay variabilidad aleatoria dentro de cada región, evitando una inicialización demasiado uniforme.

4.5. Cálculo del Vector de Residuales $\mathbf{F}(\mathbf{U})$

El vector de residuales $\mathbf{F}(\mathbf{U})$ evalúa qué tan bien la solución actual satisface las ecuaciones discretizadas. Para cada nodo (i, j) se calcula el residual correspondiente.

4.5.1. Ecuación Discretizada Central

Para nodos internos que no están en fronteras ni bloques, la ecuación de momento discretizada con diferencias finitas centradas es:

$$v_{i,j}^2 = \frac{1}{4} (v_{i,j+1} + v_{i,j-1} + v_{i+1,j} + v_{i-1,j} - 4v_{i,j}[v_{i,j+1} - v_{i,j-1}] - 4V_y[v_{i-1,j} - v_{i+1,j}])$$

donde $V_y = -0.000005$ es la componente de velocidad en dirección y (constante).

El residual para este nodo se define como:

$$F_{i \times 52 + j} = -v_{i,j} + \frac{1}{4} (v_{i,j+1} + v_{i,j-1} + v_{i+1,j} + v_{i-1,j} - 4v_{i,j}[v_{i,j+1} - v_{i,j-1}] - 4V_y[v_{i-1,j} - v_{i+1,j}])$$

En forma vectorial, si $X_{\text{idx}} = v_{i,j}$ es el valor en el índice lineal $\text{idx} = i \times 52 + j$:

$$F_{\text{idx}} = -X_{\text{idx}} + \frac{1}{4} \left(X_{\text{idx}+1} + X_{\text{idx}-1} + X_{\text{idx}+52} + X_{\text{idx}-52} - 4X_{\text{idx}}[X_{\text{idx}+1} - X_{\text{idx}-1}] - 4V_y[X_{\text{idx}-52} - X_{\text{idx}+52}] \right)$$

$$F_{i \times 52 + j} = -X_{i \times 52 + j} + \frac{1}{4} \left(X_{i \times 52 + (j+1)} + X_{i \times 52 + (j-1)} + X_{(i+1) \times 52 + j} + X_{(i-1) \times 52 + j} - 4X_{i \times 52 + j} [X_{i \times 52 + (j+1)} - X_{i \times 52 + (j-1)}] - 4V_y [X_{(i+1) \times 52 + j} - X_{(i-1) \times 52 + j}] \right)$$

Figura 3: Ecuación del residual $F_{i \times 52 + j}$ para cada punto de la malla. Esta ecuación representa el balance entre difusión y convección discretizado.

4.5.2. Condiciones de Frontera

Las condiciones de frontera se imponen directamente en el vector de residuales:

Frontera Izquierda (Inlet $\mathbf{F}$): $j = 0$

- Para $i < i_{\text{bloque_sup}}$: $v_{i,0} = v_0 = 1$

$$F_{i \times 52 + 0} = v_{i,0} - 1 = 0 \quad \Rightarrow \quad F_{\text{idx}} = 0$$

- Para $i \geq i_{\text{bloque_sup}}$: $v_{i,0} = 0$

$$F_{i \times 52 + 0} = v_{i,0} = 0 \quad \Rightarrow \quad F_{\text{idx}} = 0$$

Frontera Superior (Surface G): $i = 0$

- Para $j < 37$: $v_{0,j} = v_0 = 1$

$$F_{0 \times 52+j} = v_{0,j} - 1 = 0 \quad \Rightarrow \quad F_{\text{idx}} = 0$$

- Para $j \geq 37$: $v_{0,j} = 0$

$$F_{0 \times 52+j} = v_{0,j} = 0 \quad \Rightarrow \quad F_{\text{idx}} = 0$$

Frontera Derecha (Outlet A): $j = 51$

$$v_{i,51} = 0 \quad \Rightarrow \quad F_{i \times 52+51} = 0$$

Frontera Inferior (E): $i = 6$

$$v_{6,j} = 0 \quad \Rightarrow \quad F_{6 \times 52+j} = 0$$

Bloques Sólidos (B): Para cualquier nodo (i, j) que esté dentro de un bloque:

$$v_{i,j} = 0 \quad \Rightarrow \quad F_{i \times 52+j} = 0$$

Los bloques están definidos por:

- **Bloque Superior:** $(37 \leq j \leq 50)$ y $(4 \leq i \leq 5)$
- **Bloque Inferior:** $(22 \leq j \leq 29)$ y $(1 \leq i \leq 2)$

4.6. Cálculo de la Matriz Jacobiana $J(\mathbf{U})$

La matriz Jacobiana es la derivada del vector de residuales respecto al vector de incógnitas:

$$J_{pq} = \frac{\partial F_p}{\partial X_q}$$

Es una matriz de dimensión 364×364 que es mayormente dispersa (sparse).

4.6.1. Derivadas Parciales para Nodos Internos

Para cualquier punto diferente a frontera, las derivadas parciales del residual son:

$$\frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+j}} = -1 - X_{i \cdot 52+(j+1)} + X_{i \cdot 52+(j-1)} \quad (4)$$

$$\frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+(j+1)}} = \frac{1}{4} - X_{i \cdot 52+j} \quad (5)$$

$$\frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+(j-1)}} = \frac{1}{4} + X_{i \cdot 52+j} \quad (6)$$

$$\frac{\partial F_{i \cdot 52+j}}{\partial X_{(i+1) \cdot 52+j}} = \frac{1}{4} - V_y \quad (7)$$

$$\frac{\partial F_{i \cdot 52+j}}{\partial X_{(i-1) \cdot 52+j}} = \frac{1}{4} + V_y \quad (8)$$

4.6.2. Derivadas Parciales para Nodos de Frontera

Para puntos en frontera o dentro de bloques:

$$\frac{\partial F_k}{\partial X_k} = 1$$

4.6.3. Estructura de la Matriz Jacobiana

La matriz Jacobiana tiene la siguiente estructura dispersa, donde las filas correspondientes a fronteras tienen 1 en la diagonal, y las filas interiores tienen 5 entradas no nulas:

$$J = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \cdots & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & \cdots & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & \ddots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+(j-1)}} & \frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+j}} & \frac{\partial F_{i \cdot 52+j}}{\partial X_{i \cdot 52+(j+1)}} & \cdots \\ \vdots & \vdots & \vdots & \cdots & 0 & 0 & \cdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 1 \end{pmatrix}$$

4.6.4. Mapa de calor del Jacobiano

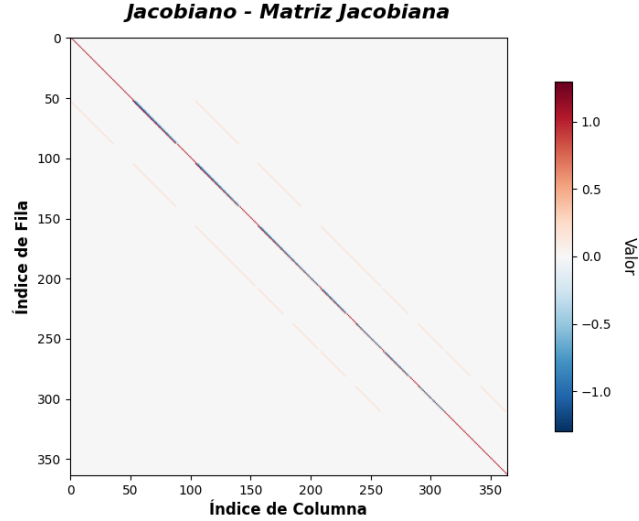


Figura 4: Mapa de calor de la matriz Jacobiana (364×364). Se observa la estructura dispersa con la diagonal dominante (azul/rojo) y las bandas secundarias correspondientes a los vecinos en la malla.

4.7. Proceso Iterativo de Newton-Raphson

El método de Newton-Raphson para sistemas no lineales se basa en la siguiente ecuación de iteración:

$$\boxed{X_{i+1} = X_i + H}$$

donde el incremento H se obtiene resolviendo el sistema lineal $J \cdot H = -F(X)$. Calcular la inversa de la Jacobiana directamente es costoso y puede introducir errores numéricos; en su lugar, se utiliza un método iterativo para resolver el sistema lineal.

El algoritmo iterativo se describe a continuación:

Algorithm 1 Método de Newton-Raphson para el Sistema No Lineal

```

1: Inicializar  $\mathbf{U}^{(0)}$  con la estrategia informada
2: Establecer tolerancias:  $\varepsilon_F = 10^{-10}$ ,  $\delta_{\text{step}} = 10^{-8}$ 
3: Establecer número máximo de iteraciones:  $k_{\text{máx}} = 200$ 
4:  $k \leftarrow 0$ 
5:  $\mathbf{U}_{\text{prev}} \leftarrow \mathbf{U}^{(0)}$ 
6: while  $k < k_{\text{máx}}$  do
7:    $k \leftarrow k + 1$ 
8:   Calcular vector de residuales:  $\mathbf{F}(\mathbf{U}^{(k-1)})$ 
9:   Calcular norma del residual:  $\|\mathbf{F}\| = \|\mathbf{F}(\mathbf{U}^{(k-1)})\|_2$ 
10:  Imprimir: Iteración  $k$ ,  $\|\mathbf{F}\|$ 
11:  if  $\|\mathbf{F}\| < \varepsilon_F$  then
12:    Convergencia alcanzada (Criterio 1: Norma del residual)
13:    break
14:  end if
15:  Calcular matriz Jacobiana:  $J(\mathbf{U}^{(k-1)})$ 
16:  Resolver sistema lineal:  $J(\mathbf{U}^{(k-1)}) \cdot \Delta \mathbf{U} = -\mathbf{F}(\mathbf{U}^{(k-1)})$ 
17:  Actualizar solución:  $\mathbf{U}^{(k)} = \mathbf{U}^{(k-1)} + \Delta \mathbf{U}$ 
18:  Calcular cambio:  $\varepsilon = \|\mathbf{U}^{(k)} - \mathbf{U}_{\text{prev}}\|_2$ 
19:  Imprimir: Epsilon (cambio) =  $\varepsilon$ 
20:  if  $\varepsilon < \delta_{\text{step}}$  then
21:    Convergencia alcanzada (Criterio 2: Cambio pequeño entre iteraciones)
22:    break
23:  end if
24:   $\mathbf{U}_{\text{prev}} \leftarrow \mathbf{U}^{(k)}$ 
25: end while
26: if  $k = k_{\text{máx}}$  then
27:  No convergió (Criterio 3: Número máximo de iteraciones alcanzado)
28: end if
29: return  $\mathbf{U}^{(k)}$ 

```

4.8. Criterios de Parada

La iteración se detiene cuando se cumple una de estas condiciones:

1. **(C1) Norma del Residual:** La norma euclidiana del vector de residuales es suficientemente

pequeña:

$$\|\mathbf{F}(\mathbf{U}^{(k)})\|_2 = \sqrt{\sum_{i=1}^{364} F_i^2} < \varepsilon_F = 10^{-10}$$

Este criterio verifica que todas las ecuaciones estén prácticamente satisfechas.

2. **(C2) Epsilon (Cambio entre Iteraciones):** La solución deja de cambiar significativamente:

$$\|\Delta \mathbf{U}^{(k)}\|_2 = \|\mathbf{U}^{(k)} - \mathbf{U}^{(k-1)}\|_2 < \delta_{\text{step}} = 10^{-8}$$

Este criterio detecta cuando el método ha alcanzado un punto estacionario (aunque no necesariamente la solución exacta).

3. **(C3) Límite de Iteraciones:** Se alcanza el número máximo de iteraciones:

$$k \geq k_{\text{máx}} = 200$$

Este criterio evita bucles infinitos en caso de no convergencia.

5 Solución del Sistema Lineal: Elección del Método Iterativo

En cada iteración del método de Newton-Raphson, debemos resolver el sistema lineal:

$$J(\mathbf{U}^{(k)}) \cdot H = -\mathbf{F}(\mathbf{U}^{(k)})$$

donde $H = \Delta \mathbf{U}^{(k)}$ es el incremento que buscamos. Este sistema tiene dimensión 364×364 (para nuestra malla de 52×7 nodos), lo que hace costoso el cálculo directo de la inversa. Por ello, analizamos diversos métodos iterativos para sistemas lineales.

5.1. Marco Teórico: La Teoría de Separación (Splitting)

El principio rector de los métodos iterativos estacionarios es convertir un problema difícil ($Ax = b$) en una secuencia de problemas fáciles [2]. La idea es **separar la matriz A** en dos partes: una matriz Q (fácil de invertir) y el resto ($Q - A$).

Derivación: Partimos de $Ax = b$ y escribimos $A = Q - (Q - A)$:

$$\begin{aligned} Ax = b &\implies (Q - (Q - A))x = b \\ &\implies Qx - (Q - A)x = b \end{aligned}$$

Creamos una fórmula iterativa donde el lado izquierdo es el “futuro” ($k + 1$) y el derecho el “presente” (k):

$$Qx^{(k+1)} = (Q - A)x^{(k)} + b$$

Multiplicando por Q^{-1} :

$$\boxed{x^{(k+1)} = (I - Q^{-1}A)x^{(k)} + Q^{-1}b = Gx^{(k)} + c} \quad (9)$$

donde $G = I - Q^{-1}A$ es la **matriz de iteración**. La elección de Q determina el método.

5.1.1. Teorema de Convergencia

Un método iterativo de la forma (9) converge para cualquier punto inicial si y solo si:

$$\boxed{\rho(G) < 1}$$

donde $\rho(G) = \max |\lambda_i|$ es el **radio espectral** (el mayor valor absoluto de los valores propios de G).

5.2. Métodos Analizados y Resultados

Descomponemos la matriz A en sus componentes: Diagonal (D), Triangular Inferior estricta ($-L$) y Triangular Superior estricta ($-U$), de modo que $A = D - L - U$.

5.2.1. Método de Richardson

Es la forma más simple de iteración. Elegimos $Q = I$ (la matriz identidad).

Fórmula de actualización:

$$x^{(k+1)} = x^{(k)} + (b - Ax^{(k)}) = x^{(k)} + r^{(k)}$$

donde $r^{(k)} = b - Ax^{(k)}$ es el **residuo**. La idea es: “Toma tu estimación actual y súmale el error que tienes”.

Resultado para nuestra Jacobiana:

$$\rho(G_{\text{Richardson}}) \approx 1.02 > 1 \implies \text{No converge}$$

5.2.2. Método de Jacobi

Elegimos $Q = D$ (la diagonal de A), que es trivial de invertir: $(D^{-1})_{ii} = 1/a_{ii}$.

Fórmula de actualización:

$$x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$$

Por componentes, cada variable x_i se calcula usando solo valores del paso anterior:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Requisito: Converge garantizadamente si A es **diagonalmente dominante estricta**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i$$

Resultado para nuestra Jacobiana:

- La Jacobiana **NO** es diagonalmente dominante
- $\rho(G_{\text{Jacobi}}) \approx 1.49 > 1 \implies$ **No converge**

5.2.3. Método de Gauss-Seidel

Mejora de Jacobi: si ya calculamos $x_1^{(k+1)}$, ¿por qué usar el viejo $x_1^{(k)}$ para calcular $x_2^{(k+1)}$? Elegimos $Q = D - L$ (diagonal + triángulo inferior).

Fórmula de actualización:

$$x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$$

Requisito: También requiere dominancia diagonal o que A sea simétrica definida positiva.

Resultado para nuestra Jacobiana:

$$\rho(G_{\text{Gauss-Seidel}}) \approx 2.22 > 1 \implies \text{No converge}$$

5.2.4. Método de Gradiente Descendente

Este método cambia el paradigma: en lugar de álgebra lineal, usa **cálculo**. Si la matriz A es Simétrica y Definida Positiva (SDP), resolver $Ax = b$ equivale a minimizar la función cuadrática:

$$f(x) = \frac{1}{2}x^T Ax - b^T x, \quad \nabla f(x) = Ax - b$$

Demostración: El mínimo ocurre cuando $\nabla f(x) = Ax - b = 0$, es decir, $Ax = b$.

Algoritmo: Desde un punto $x^{(k)}$, caminamos en la dirección de máximo descenso (el negativo del gradiente, que es el residuo $r^{(k)} = b - Ax^{(k)}$):

$$x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}$$

donde el paso óptimo α_k se calcula minimizando $f(x^{(k)} + \alpha r^{(k)})$:

$$\alpha_k = \frac{r^{(k)T} r^{(k)}}{r^{(k)T} A r^{(k)}}$$

Requisito: La matriz A debe ser **simétrica** ($A^T = A$) y **definida positiva** ($x^T Ax > 0, \forall x \neq 0$).

Problema: Nuestra Jacobiana J **NO** es simétrica ($J^T \neq J$), por lo que no se puede aplicar directamente.

Resultado (si se forzara):

$$\rho(G_{\text{Grad.Desc.}}) \approx 0.99992 < 1 \quad \implies \quad \text{Converge, pero muy lentamente}$$

5.3. Resumen de Métodos Analizados

La siguiente tabla resume las fórmulas de actualización de cada método y su idea clave:

Método	Fórmula	Idea Clave
Richardson	$x + (b - Ax)$	Usa el error directamente
Jacobi	$D^{-1}(L + U)x + D^{-1}b$	Actualiza con datos viejos
Gauss-Seidel	$(D - L)^{-1}Ux + (D - L)^{-1}b$	Usa datos nuevos al instante
Grad. Descendente	$x + \alpha r$	Baja por pendiente máxima
Grad. Conjugado	$x + \alpha d$ (conjugado)	Pendiente “inteligente”

Cuadro 4: Resumen de fórmulas de actualización de métodos iterativos.

La siguiente tabla muestra los resultados de convergencia para nuestra matriz Jacobiana:

Método	Radio Espectral $\rho(G)$	¿Converge?	Razón del fallo
Richardson	≈ 1.02	No	$\rho > 1$
Jacobi	≈ 1.49	No	No hay dominancia diagonal
Gauss-Seidel	≈ 2.22	No	No hay dominancia diagonal
Grad. Descendente	≈ 0.99992	Muy lento	J no es simétrica

Cuadro 5: Comparación de métodos iterativos para la matriz Jacobiana.

5.4. El Problema con la Matriz Jacobiana

Nuestra matriz Jacobiana $J \in \mathbb{R}^{364 \times 364}$ presenta las siguientes características:

- **No es simétrica:** $J^T \neq J$
- **No es definida positiva:** Tiene valores propios negativos (algunos: $-3.38, -3.26, -3.14, \dots$)
- **No es diagonalmente dominante**
- **Número de condición:** $\kappa(J) \approx 24.02$

Estas propiedades hacen que los métodos iterativos clásicos **no converjan** o converjan demasiado lento. La solución es multiplicar por J^T para obtener $J^T J$ que sí es simétrica y definida positiva, permitiendo usar Gradiente Conjugado.

5.5. La Solución: Transformación a Sistema Simétrico Definido Positivo

Para aplicar el método de Gradiente Conjugado (que requiere matriz SDP), transformamos el sistema original multiplicando ambos lados por J^T :

Sistema original:

$$J \cdot H = -F(X)$$

Sistema transformado:

$$J^T J \cdot H = -J^T F(X)$$

5.5.1. Propiedades de $A = J^T J$

1. **Simétrica:** $(J^T J)^T = J^T (J^T)^T = J^T J \checkmark$
2. **Definida Positiva:** Para cualquier $x \neq 0$:

$$x^T (J^T J) x = (Jx)^T (Jx) = \|Jx\|^2 \geq 0$$

y es estrictamente positivo si J tiene rango completo (todas las ecuaciones son independientes).

3. **Invertible:** Si $\text{rank}(J) = n$, entonces $J^T J$ es invertible.

5.5.2. Efecto en el Número de Condición

El **número de condición** $\kappa(A) = \|A\|_2 \cdot \|A^{-1}\|_2$ mide la sensibilidad del sistema a errores numéricos [?]:

- Número de condición de J : $\kappa(J) \approx 24.02$
- Número de condición de $J^T J$: $\kappa(J^T J) \approx 577.08 - 689.39$

Observación: El número de condición aumenta al cuadrado ($\kappa(J^T J) \approx \kappa(J)^2$), lo que hace el sistema más sensible a errores numéricos. Sin embargo, ahora **sí podemos** aplicar Gradiente Conjugado, que tiene convergencia garantizada.

5.6. Método de Gradiente Conjugado

El Gradiente Conjugado es el “santo grial” de los métodos iterativos para matrices SDP [?]. En lugar de seguir la dirección del gradiente (que causa zigzag), busca direcciones **A -conjugadas** (ortogonales respecto a la matriz A). La matriz A debe cumplir: $A \in \mathbb{R}^{n \times n}$, $A^T = A$, y $\mathbf{x}^T A \mathbf{x} > 0$ para todo $\mathbf{x} \neq 0$.

5.6.1. Definición de Direcciones Conjugadas

Dos vectores d_i y d_j son **conjugados** respecto a A si:

$$d_i^T A d_j = 0, \quad i \neq j$$

Esto significa que moverse en la dirección d_j no “arruina” la minimización que ya hicimos en d_i .

5.6.2. Algoritmo de Gradiente Conjugado

Algorithm 2 Gradiente Conjugado para $AH = b$

```

1:  $H^{(0)} = 0$  (estimación inicial)
2:  $r^{(0)} = b - AH^{(0)} = b$  (residuo inicial)
3:  $d^{(0)} = r^{(0)}$  (dirección inicial = gradiente)
4: for  $k = 0, 1, 2, \dots$  hasta convergencia do
5:    $\alpha_k = \frac{r^{(k)T} r^{(k)}}{d^{(k)T} A d^{(k)}}$  {Tamaño del paso}
6:    $H^{(k+1)} = H^{(k)} + \alpha_k d^{(k)}$  {Actualizar solución}
7:    $r^{(k+1)} = r^{(k)} - \alpha_k A d^{(k)}$  {Actualizar residuo}
8:   if  $\|r^{(k+1)}\| < \varepsilon \|r^{(0)}\|$  then
9:     Convergencia alcanzada
10:    break
11:  end if
12:   $\beta_k = \frac{r^{(k+1)T} r^{(k+1)}}{r^{(k)T} r^{(k)}}$  {Coeficiente de conjugación}
13:   $d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}$  {Nueva dirección conjugada}
14: end for

```

5.6.3. Ventajas del Gradiente Conjugado

- **Convergencia garantizada:** Para una matriz SDP de tamaño n , converge en máximo n iteraciones (teóricamente exacto).
- **Sin zigzag:** Las direcciones conjugadas evitan repetir trabajo.
- **Bajo costo por iteración:** Solo requiere multiplicaciones matriz-vector.

5.7. Criterios de Parada del Gradiente Conjugado

En nuestra implementación, el Gradiente Conjugado interno se detiene cuando:

1. Norma del residuo relativa:

$$\|r^{(k)}\|_2 < \varepsilon \cdot \|r^{(0)}\|_2, \quad \varepsilon = 10^{-10}$$

2. Límite de iteraciones: $k \geq 364$ (dimensión del sistema)

3. Evitación de división por cero: Si $|d^{(k)T} A d^{(k)}| < 10^{-30}$, indica que la matriz está mal condicionada.

5.8. Resultados de la Implementación

Con la transformación $J^T J$ y el método de Gradiente Conjugado, obtuvimos los siguientes resultados:

Métrica	Valor
Iteraciones promedio de Gradiente Conjugado (por paso Newton)	~ 100
Iteraciones promedio de Newton-Raphson hasta convergencia	~ 116
Número de condición de J	$24.02 - 1184.53$
Número de condición de $J^T J$	$577.08 - 689.39$
Criterio de convergencia alcanzado	Epsilon $< 10^{-8}$
Valor máximo final en la malla	1.0000
Valor mínimo final en la malla	~ 0

Cuadro 6: Resultados de convergencia del método implementado.

5.9. Implementación y Resultados

El método fue implementado en Python usando las bibliotecas NumPy para álgebra lineal y Matplotlib para visualización. El código incluye:

- Clase **Malla**: Gestiona la geometría, condiciones de frontera y valores iniciales
- Clase **Bloque**: Define los obstáculos sólidos
- Clase **Vector**: Implementa el cálculo de $\mathbf{F}(\mathbf{U})$, $J(\mathbf{U})$, la transformación $J^T J$ y el método de Gradiente Conjugado para resolver el sistema lineal en cada iteración
- Función principal que ejecuta el ciclo de Newton-Raphson con monitoreo en tiempo real del número de condición, valores propios y convergencia del Gradiente Conjugado interno

El sistema lineal $J \cdot H = -F(X)$ se resuelve mediante la transformación $J^T J \cdot H = -J^T F(X)$, que permite aplicar el método de Gradiente Conjugado gracias a que $J^T J$ es simétrica y definida positiva.

5.9.1. Mapa de calor con valores numéricos

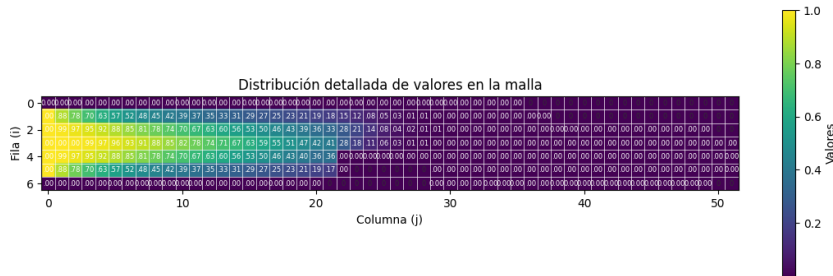


Figura 5: Malla de valores finales después de la convergencia del método de Newton-Raphson. Se observa la distribución de velocidades resultante.

5.9.2. Mapa de calor

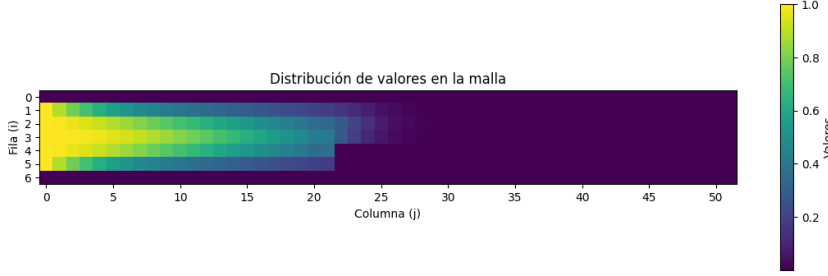


Figura 6: Malla de valores finales después de la convergencia del método de Newton-Raphson. Se observa la distribución de velocidades resultante.

5.10. Refinamiento Visual mediante Interpolación

Para mejorar la calidad de visualización en la malla reducida (50×5), se aplicó interpolación mediante splines cúbicos bidimensionales [3]. Esta técnica permite generar representaciones suavizadas del campo de velocidades sin modificar los valores calculados originalmente.

5.10.1. Metodología de Interpolación

Se utiliza la función `RectBivariateSpline` de SciPy con los siguientes parámetros:

- **Orden de los splines:** $k_x = k_y = 3$ (cúbicos)
- **Factor de suavizado:** $s = 0$ (interpolación exacta en los nodos)
- **Factor de refinamiento:** $f = 10$ (genera $10\times$ más puntos por eje)

Esto resulta en una malla de visualización de $(50 - 1) \times 10 + 1 = 491$ puntos en dirección x y $(5 - 1) \times 10 + 1 = 41$ puntos en dirección y , proporcionando una representación visual continua del campo de velocidades.

5.10.2. Visualización de la Función de Interpolación

Para comprender mejor la naturaleza de la interpolación spline, se generan dos visualizaciones complementarias que muestran el comportamiento de la función escalar $S(x, y)$ construida a partir de los valores nodales:

Superficie 3D del Spline. La función de interpolación $S : [0, 49] \times [0, 4] \rightarrow \mathbb{R}$ se puede visualizar como una superficie tridimensional donde:

- Los ejes horizontales (x, y) representan las coordenadas de la malla
- El eje vertical representa el valor de velocidad $u(x, y)$
- La superficie es C^2 -continua (derivadas segundas continuas) gracias al orden cúbico del spline

Superficie de Interpolación - Spline Cúbico Bidimensional

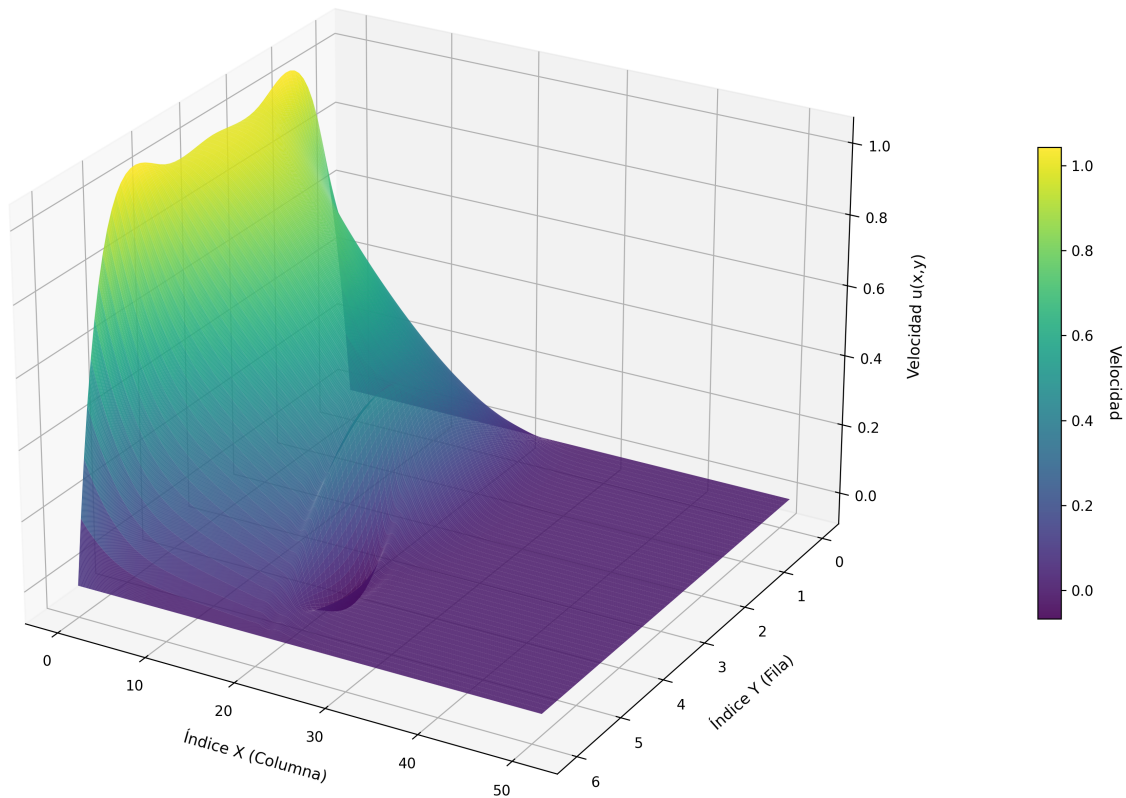


Figura 7: Superficie 3D de la función de interpolación spline cúbico bidimensional. Se observa claramente la variación del campo de velocidades: valores altos en la entrada (izquierda superior) que decrecen gradualmente hacia la salida (derecha inferior), con valles correspondientes a las zonas de los bloques.

Cortes y Perfiles de Velocidad. Para analizar el comportamiento del spline en detalle, se trazan cortes de la función a lo largo de líneas de y constante (perfiles horizontales) y x constante (perfiles verticales):

Cortes de la Función de Interpolación Spline

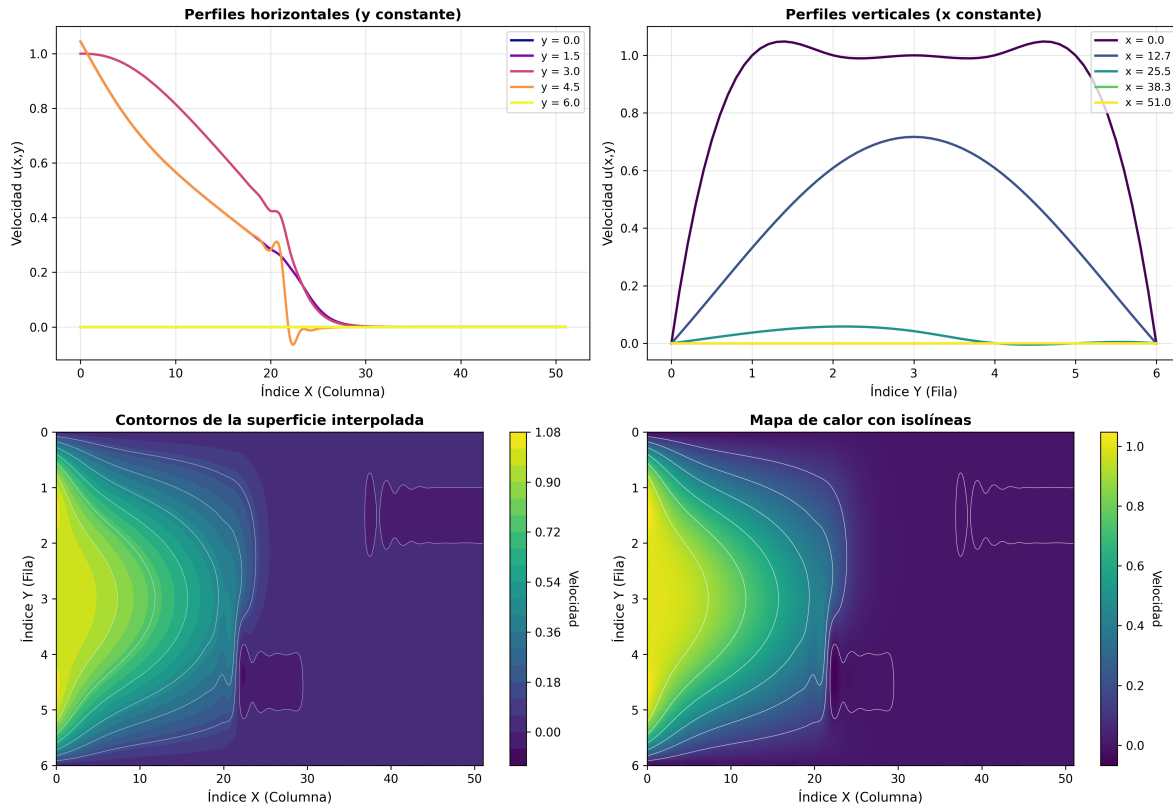


Figura 8: Análisis de la función de interpolación mediante cortes. **Superior izquierda:** Perfiles horizontales mostrando la variación de velocidad a diferentes alturas. **Superior derecha:** Perfiles verticales mostrando la distribución en diferentes secciones del canal. **Inferior izquierda:** Curvas de nivel (contornos) de la superficie. **Inferior derecha:** Mapa de calor con isolíneas superpuestas.

Los perfiles permiten identificar:

- La suavidad de la interpolación sin oscilaciones espurias
- La monotonía de los perfiles en regiones alejadas de los obstáculos
- Las perturbaciones localizadas causadas por la presencia de los bloques

5.10.3. Resultado Final Suavizado

Aplicando la interpolación a toda la malla, se obtiene el campo de velocidades refinado:

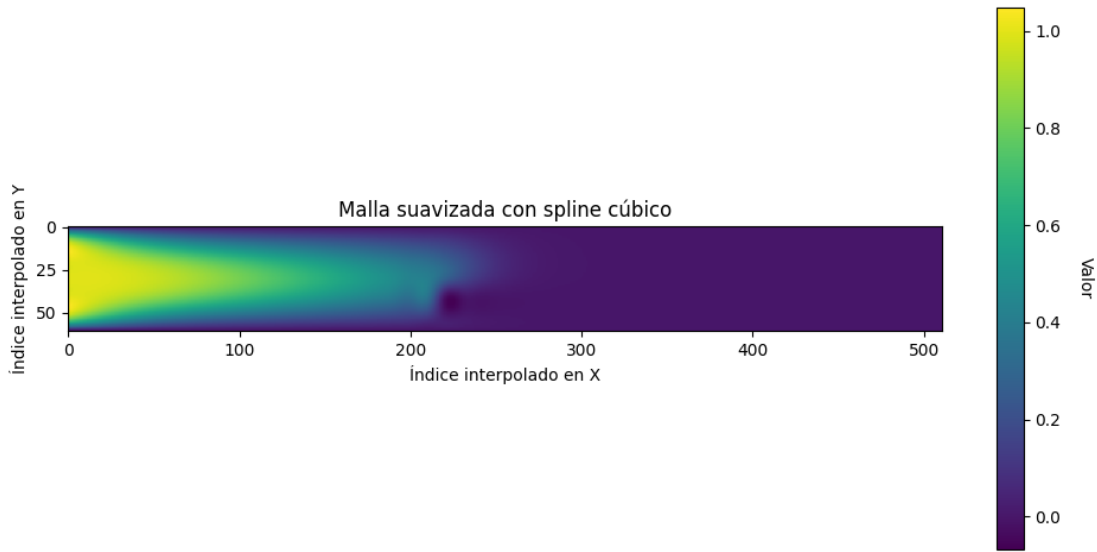


Figura 9: Campo de velocidades suavizado mediante interpolación con splines cúbicos. La visualización refinada muestra claramente la distribución de velocidades desde la entrada (izquierda, valores altos en amarillo) hasta la salida (derecha, valores bajos en azul), con las zonas de los bloques en velocidad cero.

5.11. Observaciones sobre la Convergencia

Durante la ejecución del algoritmo se observa:

- La norma del residual $\|\mathbf{F}\|$ disminuye rápidamente en las primeras iteraciones (convergencia cuadrática característica de Newton-Raphson).
- El Gradiente Conjugado interno típicamente converge en ~ 100 iteraciones, aunque ocasionalmente detecta matrices mal condicionadas.
- La inicialización informada (valores decrecientes de izquierda a derecha y de arriba a abajo) mejora significativamente la convergencia.
- La transformación $J^T J$ permite usar Gradiente Conjugado a costa de aumentar el número de condición.

6 Conclusiones y Próximos Pasos

Conclusiones. Se ha establecido un marco teórico y numérico sólido para la simulación. Se formalizó la geometría, las mallas, la discretización por diferencias finitas y el catálogo completo de las ecuaciones especiales para el tratamiento de fronteras. Crucialmente, se ha definido e implementado:

- Una estrategia de inicialización informada para el método de Newton-Raphson, adaptada a la física esperada del problema.
- El cálculo completo del vector de residuales $\mathbf{F}(\mathbf{U})$ incluyendo todas las condiciones de frontera.
- El cálculo de la matriz Jacobiana $J(\mathbf{U})$ con derivadas parciales exactas.
- Tres criterios de parada robustos para garantizar la convergencia o detectar problemas.
- Implementación funcional en Python con visualización de resultados.

Próximos Pasos.

- **Validación Extendida:** Verificar la convergencia y estabilidad del método en diferentes condiciones iniciales y parámetros físicos.
- **Optimización para Mallas Grandes:** Implementar técnicas de matrices dispersas (sparse matrices) para reducir el costo computacional al escalar a $h = 1$.
- **Integración del Sistema Completo:** Levantar las simplificaciones actuales. Esto implica:
 - Discretizar e incorporar la ecuación de momento para la componente v_y .
 - Implementar un método para satisfacer la ecuación de continuidad (ej., un esquema de corrección de presión o la formulación de vorticidad-función de corriente).
- **Análisis y Escalado:** Una vez validado el modelo completo, ejecutar la simulación en la malla fina (400×40 con $h = 1$) y analizar los resultados.
- **Visualización Avanzada:** Generar gráficos de campos vectoriales, líneas de corriente y contornos de vorticidad para interpretar físicamente la interacción del flujo con los obstáculos.
- **Análisis de Sensibilidad:** Estudiar cómo varían los resultados con diferentes valores de V_y , diferentes geometrías de bloques y diferentes condiciones de frontera.

7 Repositorio del Proyecto

El código fuente completo de este proyecto está disponible públicamente en GitHub:

<https://github.com/Suflanton1713/laminar-flow-simulation>

7.1. Contenido del Repositorio

- `mall.py`: Código principal de la simulación en Python, incluyendo las clases `Bloque`, `Malla` y `Vector`, así como todas las funciones de visualización e interpolación.
- `informe.tex`: Este documento en formato L^AT_EX.
- `README.md`: Documentación completa del proyecto.
- Imágenes generadas: Visualizaciones del Jacobiano, valores iniciales, superficie 3D del spline, cortes de interpolación y campo de velocidades suavizado.
- `deprecated_initial_values/`: Carpeta con valores iniciales de pruebas anteriores.

7.2. Sobre el Modelo y los Datos Iniciales

El modelo implementado está basado en el problema propuesto en el libro de **Landau & Páez** [1] (Capítulo 4.6: Hidrodinámica). La geometría del canal, la posición de los obstáculos y los parámetros físicos son valores fijos establecidos según esta referencia.

Los **valores iniciales** para el método de Newton-Raphson son **generados automáticamente** por el código mediante un algoritmo que simula el comportamiento físico esperado del fluido: velocidades que decrecen de izquierda a derecha (pérdida de momento) y de arriba hacia abajo (perfil de velocidad). No se requiere ninguna configuración manual ni archivos de datos externos.

7.3. Documentación en el README

El archivo `README.md` contiene documentación detallada que incluye:

- **Documentación de clases**: Descripción completa de las clases `Bloque`, `Malla` y `Vector` con sus parámetros, atributos y métodos.
- **Documentación de funciones**: Explicación de todas las funciones auxiliares de análisis, visualización e interpolación.
- **Diagrama del proceso `main()`**: Flujo de ejecución paso a paso desde la definición de geometría hasta la generación de resultados suavizados.
- **Instrucciones de instalación**: Requisitos y comandos para ejecutar la simulación.
- **Descripción de resultados**: Tabla con todos los archivos de imagen generados.

Referencias

- [1] R. Landau & M. Páez. *Computational Problems for Physics: With Guided Solutions Using Python*. CRC Press, 2018. Cap. 4.6 Hidrodinámica (Navier–Stokes, viga sumergida, vorticidad).
- [2] D. Kincaid & W. Cheney. *Numerical Analysis: Mathematics of Scientific Computing*. Brooks/Cole, 3rd Edition, 2002.
- [3] P. Virtanen et al. *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17:261–272, 2020.